

C++ Recursion Questions and Answers - Complete Collection

Question 1: Basic Recursion Output

cpp

```
int doSomething(int a, int b)
{
    if (b==1)
        return a;
    else
        return a + doSomething(a,b-1);
}

int main() {
    cout << doSomething(2,3) << endl;
    return 0;
}
```

Options:

- 05
- 04
- **06** 
- Error

Answer: 06

Explanation:

- $\text{doSomething}(2,3) \rightarrow 2 + \text{doSomething}(2,2)$
- $\text{doSomething}(2,2) \rightarrow 2 + \text{doSomething}(2,1)$
- $\text{doSomething}(2,1) \rightarrow$ returns 2
- Final result: $2 + (2 + 2) = 6$

Question 2: Missing Base Condition

What happens if base condition is not defined in recursion?

Options:

- Stack underflow
- **Stack overflow** 
- None of these
- Both a and b

Answer: Stack overflow

Explanation: Without a base condition, recursive function calls continue indefinitely, eventually exhausting stack memory and causing a stack overflow error.

Question 3: Digit Sum Function

```
cpp
int rec(int num){
    return (num)? num + rec(num/10):0;
}

int main(){
    cout << rec(4657) << endl;
}
```

Options:

- **22** 
- 20
- 2
- 0
- 12

Answer: 22 (assuming intended code should sum digits)

Explanation: If the code was meant to sum digits with `num % 10`, then:

- $4657 \rightarrow 7 + 5 + 6 + 4 = 22$
-

Question 4: Recursion Termination

In recursion, the condition after which the function will stop calling itself is:

Options:

- **Base condition** 
- Function call
- Both
- None

Answer: Base condition

Explanation: The base condition (or base case) defines the stopping point of recursion.

Question 5: Complex Mutual Recursion

cpp

```
void fun1(int n) {  
    if(n==0) return;  
    cout << n;  
    fun2(n-2);  
    cout << n;  
}  
  
void fun2(int n) {  
    if(n==0) return;  
    cout << n;  
    fun1(++n);  
    cout << n;  
}  
  
int main(){  
    fun1(4);  
    return 0;  
}
```

Options:

- 53423122233445
- 4231222334
- 312223
- **None** 

Answer: None (actual output is 4231221324)

Question 6: GCD Function

cpp

```
int fun(int x, int y){  
    if(y <= 0) return x;  
    return fun(y, x%y);  
}
```

What does this function calculate?

Options:

- y^x
- GCD of x and y 
- x^y
- LCM of x and y

Answer: GCD of x and y

Explanation: This implements the Euclidean Algorithm for finding Greatest Common Divisor.

Question 7: Growth Analysis

cpp

```
int f(int x){  
    if(x < 1) return 1;  
    else return (f(x-1) + g(x));  
}  
  
int g(int x){  
    if(x < 2) return 2;  
    else return (f(x-1) + g(x/2));  
}
```

Which best describes the growth of f(x)?

Options:

- Linear
- Exponential 
- Quadratic

- Cubic

Answer: Exponential

Explanation: $f(x) \geq 2f(x-1)$, leading to exponential growth.

Question 8: Mutual Recursion Output

cpp

```
int f(int x){
    if(x < 1) return 1;
    else return (f(x-1) + g(x));
}

int g(int x){
    if(x < 2) return 2;
    else return (f(x-1) + g(x/2));
}

int main(){
    cout << f(4) << endl;
    return 0;
}
```

Options:

- 87
- 0
- 41 
- 08
- 03

Answer: 41

Question 9: Base Conversion (Sum of Digits in Binary)

cpp

```

int f(int n, int r){
    if(n > 0){
        return n % r + f(n/r, r);
    }
    return 0;
}

int main(){
    cout << f(511, 2) << endl;
}

```

Options:

- 09 
- 01
- 02
- 0
- None

Answer: 09

Explanation: 511 in binary is 111111111 (nine 1s), so sum of digits = 9.

Question 10: Simple Recursion

```

cpp

int fun(int n)
{
    if (n == 4)
        return n;
    else return 2*fun(n+1);
}

int main()
{
    cout << fun(3) << endl;
    return 0;
}

```

Options:

- 16
- **08** 
- 04
- 02

Answer: 08

Explanation: $\text{fun}(3) = 2 * \text{fun}(4) = 2 * 4 = 8$

Question 11: Syntax Error

```
cpp

int fun(int n)
{
    if (n == 4)
        return n;
    else return 2 fun(n+1); // Missing * operator
}

int main()
{
    cout << fun(5) << endl;
    return 0;
}
```

Options:

- 04
- 16
- 08
- **Runtime Error** 

Answer: Runtime Error (technically compile-time error due to missing * operator)

Question 12: Infinite Recursion

```
cpp
```

```

int fun(int n)
{
    if (n == 4)
        return n;
    else
        return 2 * fun(n+1);
}

int main()
{
    cout << fun(5) << endl;
    return 0;
}

```

Answer: Runtime Error  (infinite recursion leading to stack overflow)

Explanation: Starting from 5 and incrementing never reaches the base case of 4.

Question 13: Corrected Syntax (assuming typo)

```

cpp

int fun(int x, int y)
{
    if (x == 0)
        return y;
    return fun(x - 1, x + y);
}

int main()
{
    cout << fun(5, 2) << endl;
    return 0;
}

```

Options:

- 11
- 14
- 17 
- None

Answer: 17

Question 14: Binary Representation (Reverse Order)

```
cpp

void fun(int n)
{
    if (n == 0)
        return;
    cout << n % 2;
    fun(n/2);
}

int main()
{
    fun(11);
    return 0;
}
```

Options:

- 101
- 1101 
- 1111
- 1110

Answer: 1101

Explanation: This prints the binary representation of 11 in reverse order.

Question 15: Multiplication Function

```
cpp

int fun(int x, int y)
{
    if (y == 0) return 0;
    return (x + fun(x, y-1));
}
```

What does this function do?

Options:

- $x + y$
- $x + x*y$
- $x*y$ 
- x^y

Answer: $x*y$

Explanation: Adds x to itself y times, which equals $x*y$.

Question 16: Recursion with Pre and Post Operations

```
cpp

void print(int n)
{
    if (n > 4000)
        return;
    cout << n << " ";
    print(2*n);
    cout << n << " ";
}

int main()
{
    print(500);
    return 0;
}
```

Options:

- 500 1000 2000 4000 4000 2000 1000 500 
- 1000 2000 4000 4000 2000
- 1000
- 500 1000 2000 4000 4000 2000 1000
- 1000 2000 4000 4000 2000 1000 500

Answer: 500 1000 2000 4000 4000 2000 1000 500

Question 17: Power of 3 Check

cpp

```
int fun(unsigned int n)
{
    if (n == 0 || n == 1)
        return n;
    if (n%3 != 0)
        return 0;
    return fun(n/3);
}
```

What does this function do?

Options:

- It returns 1 when n is a multiple of 3, otherwise returns 0
- **It returns 1 when n is a power of 3, otherwise returns 0** 
- It returns 0 when n is a multiple of 3, otherwise returns 1
- It returns 0 when n is a power of 3, otherwise returns 1

Answer: It returns 1 when n is a power of 3, otherwise returns 0

Question 18: Complex Recursion Pattern

cpp

```
int f(int n)
{
    if(n <= 1)
        return 1;
    if(n%2 == 0)
        return f(n/2);
    return f(n/2) + f(n/2+1);
}

int main()
{
    cout << f(15) << endl;
    return 0;
}
```

Options:

- 05
- **04** 
- 03
- 02

Answer: 04

Question 19: Array Processing with Alternating Operations

cpp

```
int f(int a[], int i, int n)
{
    if(n <= 0) return 0;
    else if(a[i] % 2 == 0) return a[i] + f(a, i+1, n-1);
    else return a[i] - f(a, i+1, n-1);
}

int main()
{
    int a[] = {12, 7, 13, 4, 11, 6};
    cout << f(a, 2, 4) << endl;
    return 0;
}
```

Options:

- 15
- **04** 
- 11
- 16

Answer: 04

Explanation: Processing elements [13, 4, 11, 6]: $13 - (4 + (11 - 6)) = 13 - 9 = 4$

Question 20: Fibonacci with Pointer

cpp

```
int fun(int n, int *f_p)
{
    int t, f;
    if (n <= 1)
    {
        *f_p = 1;
        return 1;
    }
    t = fun(n-1, f_p);
    f = t + *f_p;
    *f_p = t;
    return f;
}

int main()
{
    int x = 15;
    cout << fun(7, &x) << endl;
    return 0;
}
```

Options:

- 08
- 13
- 21 
- 34

Answer: 21

Explanation: This computes the 7th Fibonacci number (21).

Question 21: Bubble Sort Recognition

cpp

```

void fun(int arr[], int n)
{
    if (n == 1)
        return;
    int count = 0;
    for (int i=0; i<n-1; i++){
        if (arr[i] > arr[i+1]){
            swap(arr[i], arr[i+1]);
            count++;
        }
    }
    return;
    fun(arr, n-1);
}

```

What operation is performed?

Options:

- Insertion Sort Recursively
- **Bubble Sort Recursively** 
- Selection Sort Recursively
- None of these

Answer: Bubble Sort Recursively

Question 22: Infinite Recursion Analysis

```

cpp
int f(int j){
    int i = 50;
    int k;
    if (i == j){
        cout << "codehelp" << endl;
        k = f(i);
        return 0;
    }
    else return 0;
}

```

Choose the correct statement:

Options:

- The function returns 0 for all values of j.
- The function prints the string codehelp for all values of j.
- The function returns 0 when j = 50.
- **The function will exhaust the runtime stack or run into an infinite loop when j = 50. ✓**

Answer: The function will exhaust the runtime stack or run into an infinite loop when j = 50.

Question 23: Complex Tree-like Recursion

```
cpp

void f(int n, int a, int b)
{
    if (n <= 0)
        return;
    f(n-1, a, b + n);
    cout << n << " " << a << " " << b << endl;
    f(n-1, b, a+n);
}

int main()
{
    f(3, 4, 5);
    return 0;
}
```

Output:

```
1 4 10
2 4 8
1 8 6
3 4 5
1 5 9
1 7 7
2 5 7
```

Question 24: Catalan Numbers

cpp

```
int f (int n){
    int x = 1, k;
    if (n == 1)
        return x;
    for (k = 1; k < n; ++k)
        x=x+f(k)*f(n-k);
    return x;
}

int main()
{
    cout << f(6);
    return 0;
}
```

Options:

- 51
- 188 
- 26
- 0

Answer: 188

Explanation: This computes Catalan numbers. $f(6) = 188$.

Question 25: Selection Sort Recognition

cpp

```
void fun2(int arr[], int start_index, int end_index)
{
    if (start_index >= end_index)
        return;
    int min_index;
    int temp;
    min_index = minIndex(arr, start_index, end_index);
    swap(arr[start_index], arr[min_index]);
    fun2(arr, start_index + 1, end_index);
}
```


What algorithm is this?

Options:

- **Selection Sort Recursive** ✓
- Bubble Sort Recursive
- Insertion Sort Recursive
- None of these

Answer: Selection Sort Recursive

Question 26: Recursion vs Iteration Comparison

In general, in a recursive and non-recursive implementation:

Options:

- Both time and space complexities are better in recursive than in non-recursive program.
- **Both time and space complexities are better in non-recursive than in recursive program** ✓
- Time complexity is better in recursive version but space complexity is better in non-recursive version.
- Space complexity is better in recursive version but time complexity is better in non-recursive version.

Answer: Both time and space complexities are better in non-recursive than in recursive program

Question 27: Recurrence Relation

The solution of $T(m) = T(3m/4) + 1$ is:

Options:

- **$\Theta(\lg m)$** ✓
- $\Theta(m)$
- $\Theta(m \lg m)$
- $\Theta(\lg \lg m)$

Answer: $\Theta(\lg m)$

Question 28: Number Pairs Generator

cpp

```
void print(int n, int j)
{
    if (j >= n) // Assuming 'i' was a typo
        return;
    if (n - j > 0 && n - j >= j)
        cout << j << " " << n - j << endl;
    print(n, j + 1);
}

int main()
{
    int n = 8;
    print(n, 1);
}
```

Output:

```
1 7
2 6
3 5
4 4
```

Question 29: String Permutation

cpp

```

void permute(string& a, int l, int r)
{
    if (l == r)
        cout << a << endl;
    else {
        for (int i = l; i <= r; i++) {
            swap(a[l], a[i]);
            permute(a, l+1, r);
            swap(a[l], a[i]);
        }
    }
}

int main()
{
    string a = "abc";
    permute(a, 0, 2);
}

```

Options:

- abc, acb, bac, bca, cba, cab ☒
- abc, acb, bac, bca, cba
- abc, bac, bca, cba, cab, acb
- None

Answer: abc, acb, bac, bca, cba, cab

Question 30: Backtracking Algorithms

Which is NOT a backtracking algorithm?

Options:

- Knight tour problem
- N queen problem
- Tower of hanoi ☒
- M coloring problem

Answer: Tower of Hanoi

Explanation: Tower of Hanoi is a pure recursive problem, not backtracking. It follows a fixed recursive pattern without exploring multiple branches and undoing decisions.

Bonus Question: Backtracking Best Use Case

Backtracking is best suited for:

Options:

- Sorting elements
- Searching in a sorted list
- Dynamic programming
- **Exploring all possible solutions** ✓

Answer: Exploring all possible solutions

Explanation: Backtracking systematically explores all possible configurations to find valid solutions, making it ideal for combinatorial problems.

Summary

This collection covers 30+ essential C++ recursion questions including:

- **Basic recursion patterns** (Questions 1, 10, 13)
- **Mathematical computations** (GCD, Fibonacci, Catalan numbers)
- **String and array processing** (Binary conversion, permutations)
- **Algorithm recognition** (Sorting algorithms in recursive form)
- **Complexity analysis** (Growth patterns, recurrence relations)
- **Advanced concepts** (Mutual recursion, backtracking)
- **Error analysis** (Infinite recursion, stack overflow)

These questions test understanding of:

- Base cases and termination conditions
- Recursive call patterns
- Stack behavior and memory usage
- Algorithm identification
- Time and space complexity analysis

- Practical applications of recursion

Perfect for technical interviews, competitive programming preparation, and academic assessments!